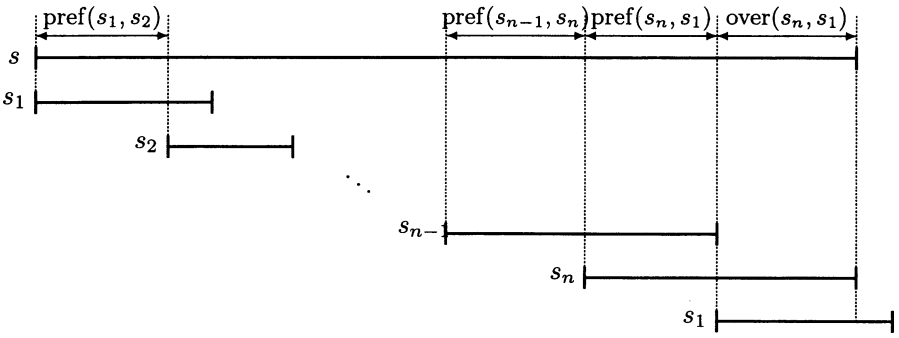
7 Shortest Superstring

In Chapter 2 we defined the shortest superstring problem (Problem 2.9) and gave a preliminary approximation algorithm using set cover. In this chapter, we will first give a factor 4 algorithm, and then we will improve this to factor 3.

7.1 A factor 4 algorithm

We begin by developing a good lower bound on OPT. Let us assume that $s_1, s_2, \ldots, s_n$ are numbered in order of leftmost occurrence in the shortest superstring, s.



Let overlap(s_i, s_j) denote the maximum overlap between s_i and s_j, i.e., the longest suffix of s_i that is a prefix of s_j. Also, let prefix(s_i, s_j) be the prefix of s_i obtained by removing its overlap with s_j. The overlap in s between two consecutive s_i's is maximum possible, because otherwise a shorter superstring can be obtained. Hence, assuming that no s_i is a substring of another, we get

$$\begin{aligned} \text{OPT} = &|\text{prefix}(s_1, s_2)| + |\text{prefix}(s_2, s_3)| + \ldots + |\text{prefix}(s_n, s_1)| \\ &+ |\text{overlap}(s_n, s_1)|. \end{aligned} \tag{7.1}$$

Notice that we have repeated s_1 at the end in order to obtain the last two terms of (7.1). This equality shows the close relation between the shortest

superstring of S and the minimum traveling salesman tour on the *prefix graph of* S, defined as the directed graph on vertex set $\{1, \ldots, n\}$ that contains an edge $i \to j$ of weight $|\text{prefix}(s_i, s_j)|$ for each i, j, $i \neq j$ (i.e., self-loops are not included). Clearly, $|\text{prefix}(s_1, s_2)| + |\text{prefix}(s_2, s_3)| + \ldots + |\text{prefix}(s_n, s_1)|$ represents the weight of the tour $1 \to 2 \to \ldots \to n \to 1$. Hence, by (7.1), the minimum weight of a traveling salesman tour of the prefix graph gives a lower bound on OPT. As such, this lower bound is not very useful, since we cannot efficiently compute a minimum traveling salesman tour.

The key idea is to lower-bound OPT using the minimum weight of a *cycle cover* of the prefix graph (a cycle cover is a collection of disjoint cycles covering all vertices). Since the tour $1 \to 2 \to \ldots \to n \to 1$ is a cycle cover, from (7.1) we get that the minimum weight of a cycle cover lower-bounds OPT.

Unlike minimum TSP, a minimum weight cycle cover can be computed in polynomial time. Corresponding to the prefix graph, construct the following bipartite graph, H. $U = \{u_1, \ldots, u_n\}$ and $V = \{v_1, \ldots, v_n\}$ are the vertex sets of the two sides of the bipartition. For each $i, j \in \{1, \ldots, n\}$ add edge (u_i, v_j) of weight $|\text{prefix}(s_i, s_j)|$. It is easy to see that each cycle cover of the prefix graph corresponds to a perfect matching of the same weight in H and vice versa. Hence, finding a minimum weight cycle cover reduces to finding a minimum weight perfect matching in H.

If $c = (i_1 \to i_2 \to \ldots i_l \to i_1)$ is a cycle in the prefix graph, let

$$\alpha(c) = \text{prefix}(s_{i_1}, s_{i_2}) \circ \ldots \circ \text{prefix}(s_{i_{l-1}}, s_{i_l}) \circ \text{prefix}(s_{i_l}, s_{i_1}).$$

Define the *weight of cycle* c, $\text{wt}(c)$, to be $|\alpha(c)|$. Notice that each string $s_{i_1}, s_{i_2}, \ldots, s_{i_l}$ is a substring of $(\alpha(c))^\infty$. Next, let

$$\sigma(c) = \alpha(c) \circ s_{i_1}.$$

Then $\sigma(c)$ is a superstring of $s_{i_1}, \ldots, s_{i_l}$.[1] In the above construction, we "opened" cycle c at an arbitrary string s_{i_1}. For the rest of the algorithm, we will call s_{i_1} the *representative string* for c. We can now state the complete algorithm:

Algorithm 7.1 (Shortest superstring – factor 4)

1. Construct the prefix graph corresponding to strings in S.
2. Find a minimum weight cycle cover of the prefix graph, $\mathcal{C} = \{c_1, \ldots, c_k\}$.
3. Output $\sigma(c_1) \circ \ldots \circ \sigma(c_k)$.

[1] This remains true even for the shorter string $\alpha(c) \circ \text{overlap}(s_l, s_1)$. We will work with $\sigma(c)$, since it will be needed for the factor 3 algorithm presented in the next section, where we use the property that $\sigma(c)$ begins and ends with a copy of s_{i_1}.